

Ansible desde Cero: Automatiza Linux y Servidores

Cursos PIT – 10ma Edicion

Instructor: Cristian Adrian Diaz Garcia

Fecha: 23 de enero de 2026



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**

CURSOS GRATUITOS
OTI UNI



TRANSFORMACIÓN
digital

SESIÓN 1

Primeros pasos con Ansible y automatización de servidores

SESIÓN 1

- Qué es Ansible y por qué se usa para automatizar infraestructura Linux
- Cómo funciona su arquitectura agentless mediante SSH
- Preparación del laboratorio con VirtualBox, Vagrant y servidores Linux
- Primeros comandos ad-hoc para ejecutar tareas remotas sin playbooks

¿Qué es Ansible?

El Problema: Administración Manual de Servidores

- Instalar paquetes uno por uno en cada servidor.
- Configurar servicios manualmente.
- Verificar estado de servicios.
- Manejar múltiples servidores se vuelve tedioso.
- Propenso a errores humanos.

Problema:
Sin automatización, la administración de servidores se vuelve tediosa y propensa a errores.



Ansible: Historia

Origen

Ansible fue creado en 2012 por Michael DeHaan con una idea central: automatizar servidores de forma simple, sin instalar agentes remotos.

- Fundación de la Empresa
- AWX y el Código Abierto
- Expansión más allá de los Servidores
- Red Hat Ansible Automation Platform



Scripts vs Ansible

Criterio	Scripts Tradicionales	Ansible (Playbooks)
Enfoque	Imperativo	Declarativo
Esfuerzo de Código	Requieren controlar muchos casos y errores manualmente.	Usa módulos listos para tareas comunes
Idempotencia	El resultado depende de cómo se escribió	Idempotente por diseño
Mantenimiento	Complejos y difíciles de reutilizar.	Playbooks altamente legibles

Ansible organiza tareas dentro de playbooks para una automatización más clara, auditable y repetible.

¿Por qué elegir Ansible?

Agentless

No requiere instalar nada en los servidores remotos

YAML legible

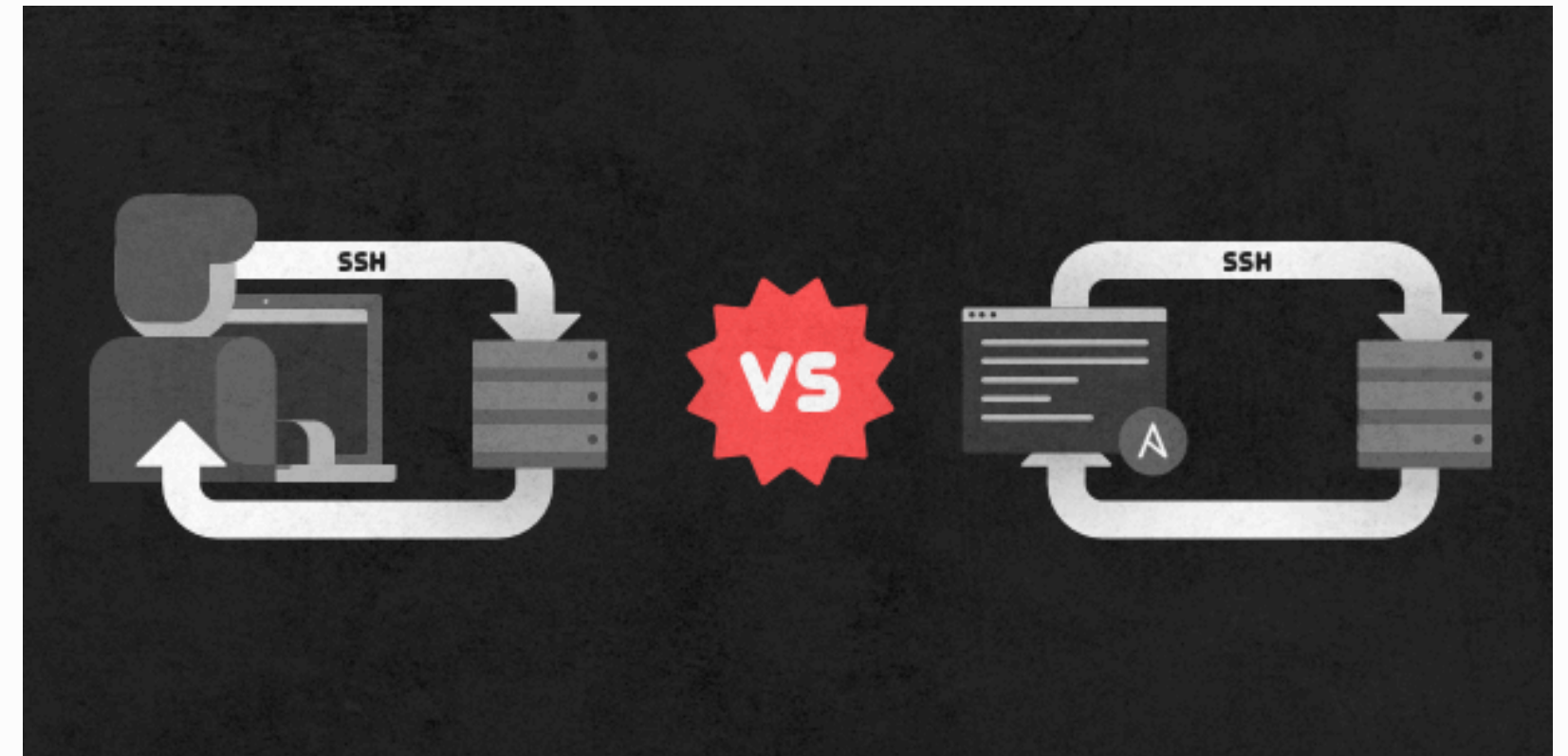
Playbooks en lenguaje simple y entendible

Idempotente

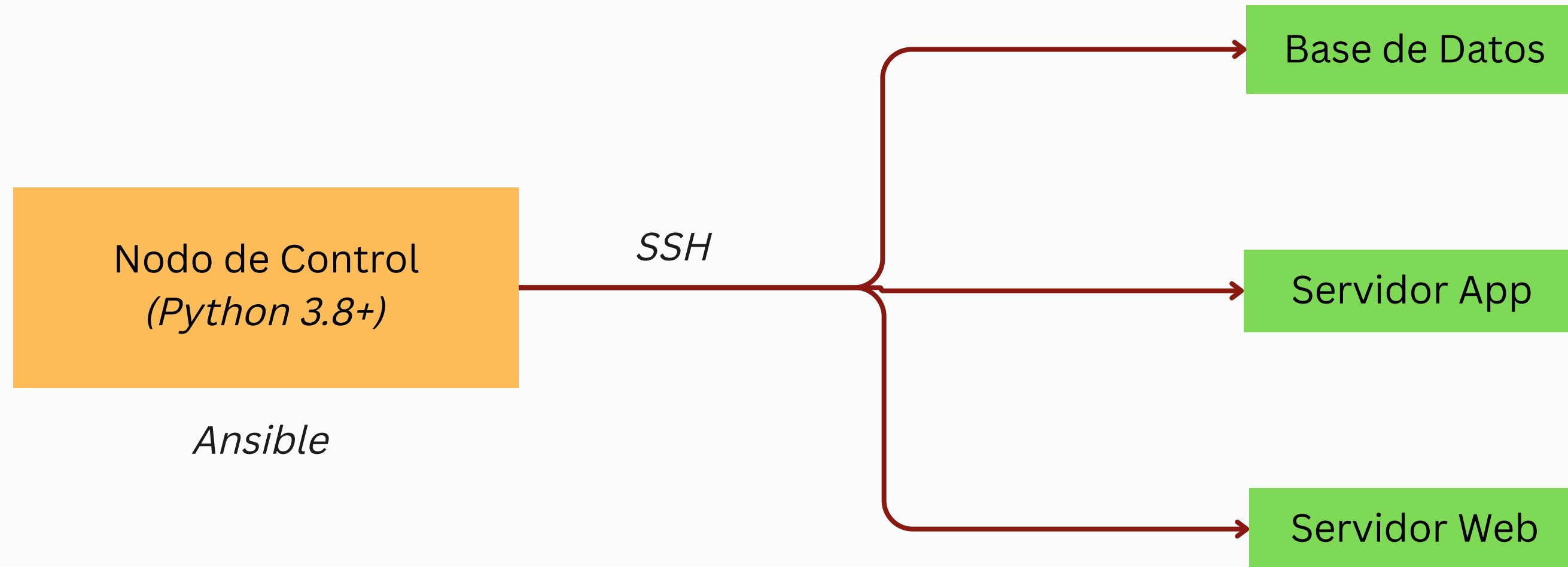
Solo aplica cambios si el sistema no está en el estado deseado

Solo necesita SSH

Conexión segura estándar ya disponible en Linux

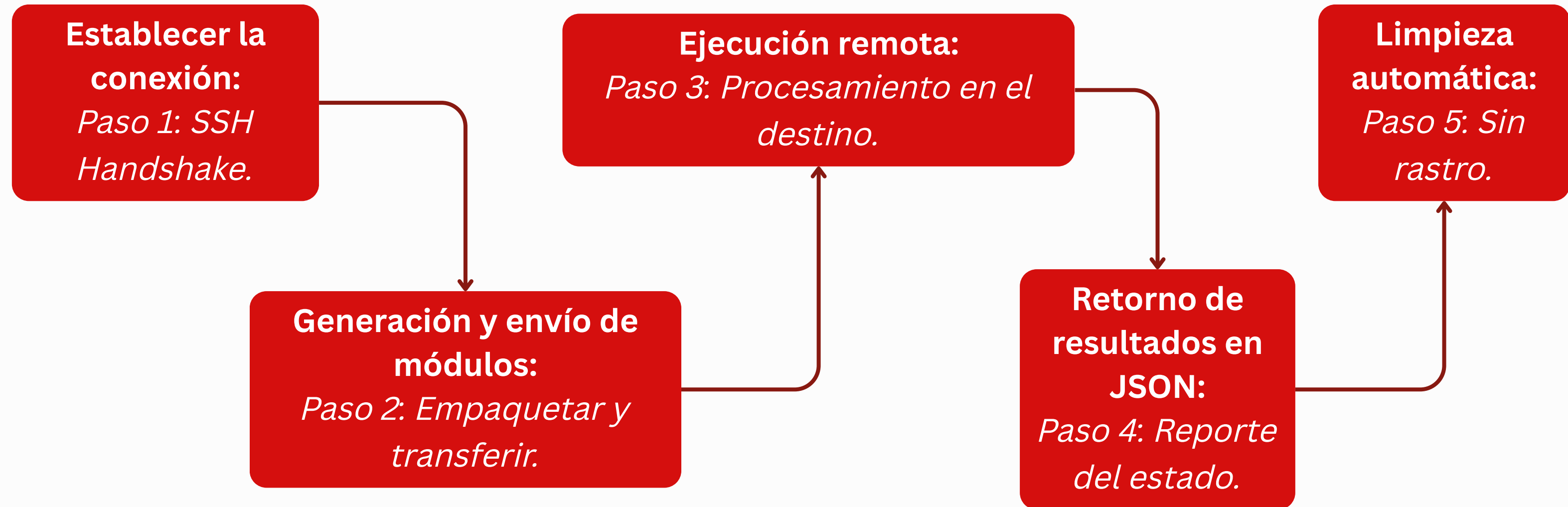


Arquitectura basica de Ansible



Modelo Push: Ansible empuja configuraciones vía SSH. Sin agentes ni demonios en los servidores = menor superficie de ataque.

El Flujo de Trabajo de Ansible



Introducción a YAML

Formatos de Datos: XML, JSON y YAML

```
<Servers>
  <Server>
    <name>Server1</name>
    <owner>John</owner>
    <created>123456</created>
    <status>active</status>
  </Server>
</Servers>
```

XML

- Muy formal, basado en etiquetas.
- Verboso para archivos de configuración.

```
{
  Servers: [
    {
      name: Server1,
      owner: John,
      created: 123456,
      status: active
    }
  ]
}
```

JSON

- Ligero, común en APIs y aplicaciones web.
- Usa llaves, corchetes y comillas con frecuencia.

```
Servers:
  - name: Server1
    owner: John
    created: 123456
    status: active
```

YAML

- Legible para humanos, usado en automatización.
- Usa indentación, listas y pares clavevalor.

Reglas Básicas de YAML

```
# Datos del servidor
servidor: web01
puerto: 80
activo: true
paquetes:
  - nginx
  - curl
```

Ideas Claves

- Ideas clave Los comentarios empiezan con #.
- La estructura depende de la indentación.
- Se recomiendan 2 espacios; no usar tabs.
- Las listas usan guion: - *item*.
- Las variables suelen ser clave: *valor*.

Reglas Básicas de YAML

```
usuario:  
  nombre: ansible  
  sudo: true  
  grupos:  
    - admins  
    - devops  
contacto: {ciudad: Lima, pais: Peru}
```

Escalares

- Valores simples: texto, números, booleanos o nulos.

Mapas

- Pares *clave: valor*, parecidos a un diccionario.

Secuencias

- Listas ordenadas de elementos.

Estructura Común de un Playbook

```
---  
- name: Preparar servidor web  
  hosts: web  
  become: true  
  tasks:  
    - name: Instalar nginx  
      apt:  
        name: nginx  
        state: present
```

- **---**: Indica el inicio del archivo YAML
- **name**: Es una etiqueta descriptiva
- **hosts**: Indica el grupo del inventario
- **become**: Permite ejecutar las tareas con privilegios elevados
- **tasks**: Lista de tareas secuenciales
- **apt**: Es el módulo de Ansible específico para gestionar paquetes en sistemas Debian/Ubuntu
- **name**: nginx: Define el nombre del paquete específico que se quiere gestionar
- **state**: Define el estado deseado del paquete.

Mini Actividad: Autobiografía en YAML

```
# Autobiografia en YAML
nombre: "Tu nombre"
contacto: {correo: "tu@correo.com", ciudad: "Lima"}
habilidades:
  - Linux
  - Git
  - Ansible
metas:
  corto_plazo: "Practicar automatizacion"
  largo_plazo: "Administrar servidores"
# Fin del documento
```

Crea un archivo ***autobiografia.yaml*** para practicar estructura e indentación.

Preparación del Laboratorio

Herramientas Necesarias para el Laboratorio

VirtualBox y Ubuntu

Usaremos una única máquina virtual como entorno aislado principal para nuestro laboratorio.

Docker + Dive Into Ansible

Dentro de la VM, Docker levantará los múltiples nodos (contenedores) que gestionaremos con Ansible.



VM Ubuntu (VirtualBox)



Contenedores (Nodos)

Instalación de Ansible por Sistema Operativo

Ubuntu / Debian

```
sudo apt update  
sudo apt install ansible -y
```

RHEL/ CentOS / Fedora

```
sudo dnf install ansible -y
```

macOS

```
brew install ansible
```

En nuestro laboratorio

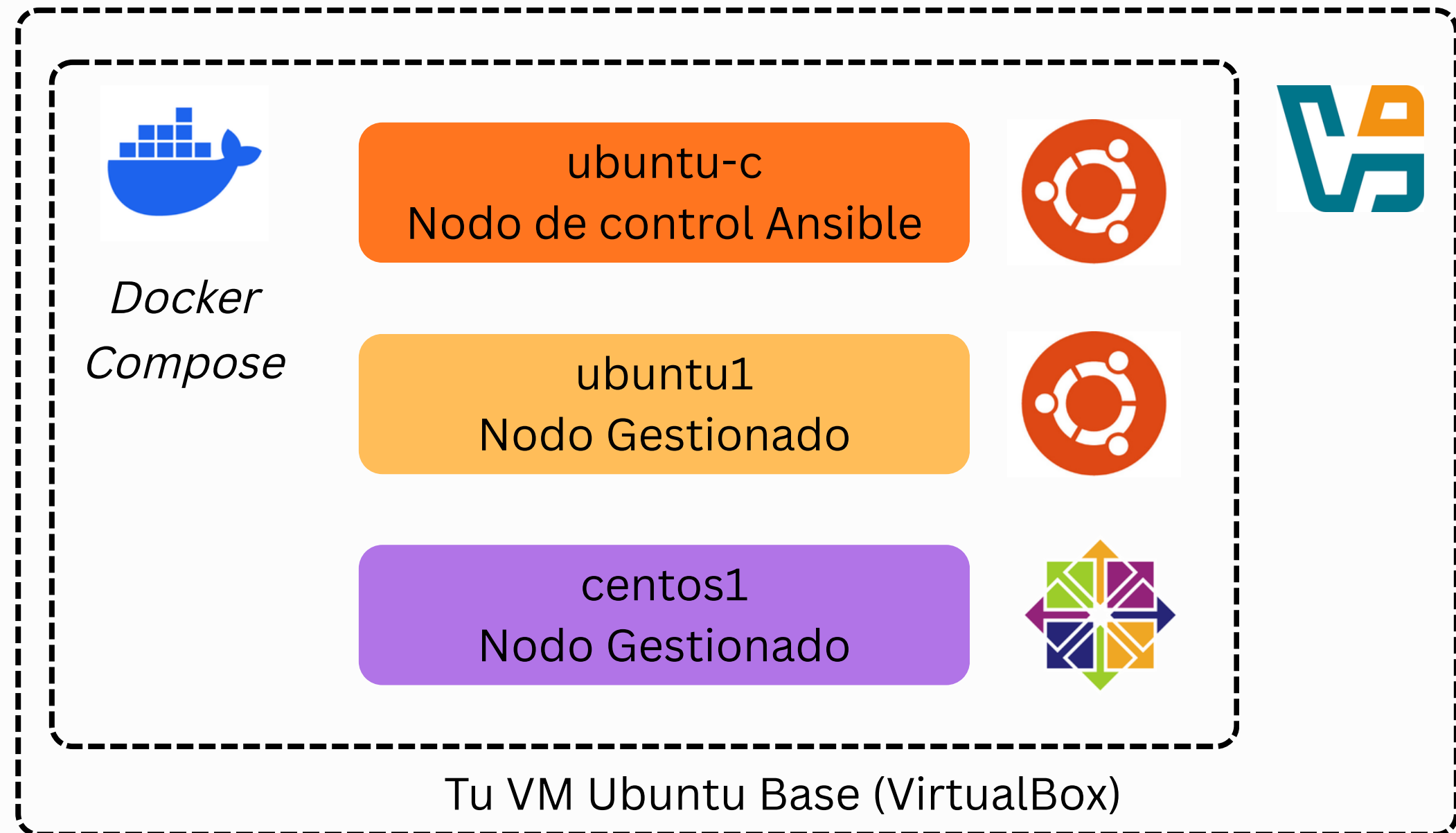
El contenedor ubuntu-c ya trae Ansible instalado; solo verificaremos con `ansible version`.



Arquitectura del Laboratorio Multi-Servidor

Objetivo

Simular un entorno real con múltiples servidores para practicar Ansible sin riesgo.



¿Por qué elegir Ansible?

Instalar Docker y Git

```
sudo apt update && sudo apt install docker.io  
docker-compose-v2 git -y
```

Clonar el repositorio

```
git clone  
https://github.com/spurin/diveintoansible-lab.git
```

Iniciar el laboratorio

```
cd diveintoansible-lab && sudo docker compose up -d
```

Importante

Esto creará contenedores interconectados. El nodo ubuntu-c actuará como tu controlador de Ansible.

Verificando el Entorno

El laboratorio de diveintoansible ya viene con **Ansible** **instalado** y las **llaves SSH preconfiguradas** entre el nodo de control y los demás nodos.

Ingresar al nodo de control y verificar:

```
sudo docker exec -it ubuntu-c bash
ansible --version
ssh ubuntu1
```

*¡Estamos listos para
empezar a usar Ansible
inmediatamente sin
configuraciones
manuales!*

Comandos Ad-Hoc

¿Qué son los Comandos Ad-Hoc?

Comandos **únicos** que se ejecutan directamente desde la terminal sin necesidad de escribir un playbook.

¿Cuándo usarlos?

- Tareas rápidas y puntuales.
- Verificar estado de servidores.
- Ejecutar un comando en múltiples máquinas al instante.

```
agrant@controller:~/ansible_project$ ansible all -m shell -a "echo $USER"
anaged2 | CHANGED | rc=0 >>
agrant
anaged1 | CHANGED | rc=0 >>
agrant
agrant@controller:~/ansible_project$ ansible all -m shell -a "uptime"
anaged1 | CHANGED | rc=0 >>
10:13:28 up 5 min,  1 user,  load average: 0.04, 0.11, 0.06
anaged2 | CHANGED | rc=0 >>
10:13:28 up 5 min,  1 user,  load average: 0.01, 0.09, 0.05
agrant@controller:~/ansible_project$ ansible all -m shell -a "cat /etc/os-release | grep -i PRETTY_NAME"
anaged2 | CHANGED | rc=0 >>
PRETTY_NAME="Ubuntu 20.04.4 LTS"
anaged1 | CHANGED | rc=0 >>
PRETTY_NAME="Ubuntu 20.04.4 LTS"
agrant@controller:~/ansible_project$ ansible all -m shell -a "touch /tmp/abc.txt"
anaged1 | CHANGED | rc=0 >>
anaged2 | CHANGED | rc=0 >>
agrant@controller:~/ansible_project$ ansible all -m shell -a "ls -l /tmp/abc.txt"
anaged2 | CHANGED | rc=0 >>
-rw-rw-r-- 1 vagrant vagrant 0 Apr 11 10:15 /tmp/abc.txt
anaged1 | CHANGED | rc=0 >>
-rw-rw-r-- 1 vagrant vagrant 0 Apr 11 10:15 /tmp/abc.txt
agrant@controller:~/ansible_project$ _
```

"Una línea, múltiples servidores."

El Archivo de Inventario

El inventario define qué servidores gestiona Ansible.

[grupos]:

Define variables globales (como el usuario SSH `ansible_user`) que se aplican automáticamente a todas las máquinas.

[all:vars]:

Agrupas tus servidores por su rol (como `web` o `db`) para aplicarles tareas específicas.

```
[web]
ubuntu1
ubuntu2

[db]
centos1
centos2

[all:vars]
ansible_user=ansible
```

*Inventario (**inventory.ini**)*

Sintaxis de Comandos Ad-Hoc

ansible

comando

web

hosts/grupo

-i inventory

inventario

-m ping

módulo

Estructura general

```
ansible <hosts> -i <inventario> -m <módulo> [-a argumentos"]
```


Ejemplos Prácticos de Comandos Ad-Hoc (1/2)

Verificar conectividad (módulo ping)

```
ansible all -i inventory.ini -m ping
```

Ejecutar un comando remoto (módulo command)

```
ansible web -i inventory.ini -m command -a "uptime"
```

Instalar un paquete (módulo apt)

```
ansible web -i inventory.ini -m apt \
-a "name=htop state=present" --become
```

Ejemplos Prácticos de Comandos Ad-Hoc (2/2)

Copiar un archivo a los servidores

```
ansible all -i inventory.ini -m copy \  
-a "src=./hola.txt dest=/tmp/hola.txt"
```

Recopilar información del sistema (facts)

```
ansible web -i inventory.ini -m setup
```

Reiniciar un servicio

```
ansible web -i inventory.ini -m service \  
-a "name=nginx state=restarted" --become
```

Módulos Más Usados en Ansible

ping (Verificar conectividad)

-m ping

command (Ejecutar comando)

-a 'whoami'

shell (Ejecutar con shell)

-a 'cat /etc/os-release'

copy (Copiar archivos)

-a 'src=... dest=...'

apt (Gestión de paquetes)

-a 'name=nginx state=present'

service (Gestión de servicios)

-a 'name=nginx state=started'

setup (Recopilar facts)

-m setup

Ansible tiene miles de módulos para casi cualquier tarea de administración.